CoderPad — User Guide

Live app: https://sigma-computing-interview-app.vercel.app

Table of Contents

1. What is CoderPad?
2. Quick Start — Conducting an Interview
3. Interviewer Guide
   - Starting a Session
   - Sending a Question to the Candidate
   - Monitoring the Candidate Live
   - Running the Candidate's Code
   - Ending a Session
4. Candidate Guide
   - Joining an Interview
   - Writing and Running Code
   - Choosing a Language
   - When the Session Ends
5. Interview Recordings & Replay
   - How Recording Works
   - Watching a Replay
   - Renaming a Session
   - Deleting a Session
   - Searching Sessions
6. Other Tools
   - Code Editor (Scratch Pad)
   - Whiteboard
   - Take-Home Mode
7. Supported Languages
8. Troubleshooting
9. FAQ

---

1. What is CoderPad?

CoderPad is a **live coding interview platform** built for Sigma Computing. It lets interviewers and candidates share a real-time code editor during technical interviews — no installation required on either side.

Key capabilities:

Feature	Detail
Live code sharing	Candidate's keystrokes appear in the interviewer's monitor view within ~1 second
Code execution	Run code server-side in 11 languages; output shown instantly
Session recording	Every interview is automatically saved and replayable for 180 days
Question bank	Push problem statements directly to the candidate's screen

Whiteboard	Freehand drawing canvas for system design discussions
Take-home mode	Send async coding problems without a live session

---

2. Quick Start — Conducting an Interview

```
Total setup time: ~30 seconds
```

Step 1 — Open the app Go to https://sigma-computing-interview-app.vercel.app in your browser.

Step 2 — Generate a session Click **Start Session** in the top toolbar.

> ***Screenshot:*** *Top toolbar showing the "Start Session" button on the right side. Two links appear in a modal dialog — a Host link and a Candidate link.*

Step 3 — Send the candidate link Copy the **Candidate link** and send it via Slack, email, or your recruiting tool.

Step 4 — Open your monitor Click **Open Host View** (or click the 👁 Monitor tab in the left sidebar).

Step 5 — Watch and evaluate The monitor shows the candidate's code updating live. Use the **Questions** panel to push a problem statement if needed.

Step 6 — End the session Click ■ **End Session** in the monitor toolbar → **Confirm end**. The candidate sees a countdown and their window closes automatically.

The session is automatically saved to your browser's 📁 Sessions panel.

---

3. Interviewer Guide

3.1 Starting a Session

1. Open https://sigma-computing-interview-app.vercel.app
2. Click **Start Session** (top-right area of the toolbar)

> ***Screenshot:*** *The link-generation dialog showing two links — "Your host link" and "Candidate link" — each with a copy button.*

Three things happen immediately:

- A unique **Session ID** (16-character hex string) is generated
- A **Host link** is created — open this to monitor the candidate
- A **Candidate link** is created — send this to your candidate

You do not need to keep the Start Session dialog open. Both links work independently.

> ***Tip:*** *You can bookmark the host link or keep it in a tab for the duration of the interview. If you accidentally close it, the same URL will reconnect you.*

3.2 Sending a Question to the Candidate

1. Click **Questions** in the top toolbar (looks like a document/list icon)
2. Browse or search the question bank
3. Click any question to select it

4. Click **Send to Candidate**

> ***Screenshot:*** *The Questions panel open over the editor, showing a list of coding problems with difficulty tags. A selected question is highlighted with a "Send to Candidate" button.*

The question appears **instantly** in the candidate's editor as a read-only description panel above their code area. The candidate cannot edit the problem statement — only their code area is editable.

> ***Note:*** *Sending a new question replaces the previous one. The candidate sees only the most recently sent question.*

3.3 Monitoring the Candidate Live

Click the 👁 **Monitor** tab in the left sidebar.

> ***Screenshot:*** *The Monitor view showing a read-only editor with the candidate's code, a green status bar reading "Candidate connected — watching live", line count, last update timestamp, and "● RECEIVING" badge when new code is arriving.*

Status bar indicators:

Indicator	Meaning
Green bar — "Candidate connected — watching live"	Candidate has their link open and has started typing
Yellow bar — "Waiting for candidate to join..."	Candidate hasn't opened the link yet
last update: Xs ago	Time since the last keystroke was received
N lines	Current line count in the candidate's editor
● RECEIVING	Code is actively being received right now

The editor is **read-only** — you can scroll through the candidate's code but cannot edit it. The view updates automatically; no refresh is needed.

3.4 Running the Candidate's Code

In the Monitor view, click ▶ **Run candidate's code**.

> ***Screenshot:*** *The Monitor toolbar with the green "▶ Run candidate's code" button. Below the editor, the Output panel is expanded showing stdout output.*

- Execution happens on the server (Railway) in an isolated sandbox
- Results appear in the **Output panel** at the bottom of the monitor
- A green ✓ badge means exit code 0 (success); a red badge shows the exit code on failure
- Execution time is shown in milliseconds

> ***Note:*** *Running code does not interfere with the candidate's view. They cannot see you ran their code unless you tell them.*

3.5 Ending a Session

In the Monitor view, click ■ **End Session** in the top-right of the toolbar.

> ***Screenshot:*** *The monitor toolbar showing the "■ End Session" button (red outline). After clicking, a "Confirm end" (red) and "Cancel" button appear side by side.*

A confirmation guard prevents accidental clicks:

1. First click → shows **Confirm end** and **Cancel**
2. Click **Confirm end** → session is marked as ended

What happens next:

- The candidate sees a full-screen "Interview Complete" overlay
- A 10-second countdown counts down visibly
- Their browser window closes automatically after 10 seconds
- If `window.close()` is blocked by the browser, the tab navigates to a blank page instead
- The candidate can also click **Close now** to dismiss immediately

The session recording is already saved to your 📁 Sessions panel regardless of when you end the session.

---

4. Candidate Guide

4.1 Joining an Interview

You will receive a link from your interviewer that looks like:

```
https://sigma-computing-interview-app.vercel.app/#iid=<sessionId>&role=candidate
```

Simply **open the link in your browser** — no sign-in, no download, no setup required.

> ***Screenshot:*** *The candidate view on load — a full Monaco code editor with a language selector, font size control, Run button, and an optional problem description panel at the top if a question was sent.*

Once open, your editor is **automatically connected** to the interviewer's session. Everything you type is shared with the interviewer in real time.

> ***Recommendation:*** *Use a modern browser (Chrome, Firefox, Edge, or Safari 16+). Avoid opening the link in a private/incognito window if you want your work to persist across accidental refreshes.*

4.2 Writing and Running Code

Type your solution in the editor. The editor supports:

- Syntax highlighting
- Auto-indentation and bracket matching
- Multi-cursor editing (Alt+Click)
- Code folding
- Find/Replace (Ctrl+F / Cmd+F)

When ready to test, click ▶ **Run** (top-right of the toolbar).

> ***Screenshot:*** *The candidate toolbar showing the Run button, language selector, font size stepper, and a "Stdin" expandable panel.*

Providing standard input (stdin): Click **Stdin** (or the input panel below the toolbar) to expand a text area. Paste or type your test input before clicking Run.

Output panel:

- stdout appears in green/white text
- stderr appears with a red "STDERR" label
- Exit code and execution time are shown in the panel header

Execution limits:

- 10 seconds maximum runtime
- 256 MB memory
- Output capped at 256 KB

4.3 Choosing a Language

Use the **language selector** dropdown in the toolbar to switch languages.

> ***Screenshot:*** *The language selector dropdown open, showing all 11 supported languages with the current language highlighted.*

Supported languages: JavaScript, TypeScript, Python, Java, C++, C, Go, Ruby, Rust, Bash, PHP.

> ***Important:*** *Switching language clears your current code. The interviewer's monitor will reflect the new language immediately.*

4.4 When the Session Ends

When the interviewer ends the session, you will see a full-screen overlay:

> ***Screenshot:*** *The "Interview Complete" overlay — a centered card with a green checkmark, "Interview Complete" heading, a message saying the session has ended, and a countdown "This window will close in 8 seconds..." with a "Close now" button.*

- The overlay appears immediately when the interviewer clicks "End Session"
- A countdown from 10 seconds begins automatically
- The window closes when the countdown reaches zero
- Click **Close now** to close immediately

If the browser cannot close the window automatically (this happens when the link was not opened via `window.open()`), the page navigates to a blank page instead.

---

5. Interview Recordings & Replay

5.1 How Recording Works

Recording is **fully automatic** — no action is required.

Every time the interviewer's polling loop detects a code change from the candidate, a snapshot is saved to the **interviewer's browser localStorage**. This happens continuously throughout the interview.

What is saved per snapshot:

- Timestamp
- Full code text at that moment
- Language
- Most recent execution output (if any)

Storage limits:

- Up to **600 snapshots** per session (~10 min of 10-second-sampled history for active coding)
- Sessions are kept for **180 days**, then auto-purged
- Recordings are stored in the **interviewer's browser only** — they are not uploaded to any server

> ***Warning:*** *Recordings are tied to the specific browser and device you used to conduct the interview. Clearing browser data, switching browsers, or using a different computer will lose the recordings.*

5.2 Watching a Replay

1. Click the 📁 **Sessions** tab in the left sidebar
2. Find the session you want to review
3. Click 📼 **Replay**

> ***Screenshot:*** *The Sessions list showing several rows, each with language badge, session name (if renamed), date, duration, and code preview. The "📼 Replay" and "✏️" and "🗑" buttons are on the right side.*

Replay controls:

> ***Screenshot:*** *The Replay view with a read-only code editor, a timeline scrubber at the bottom, Play/Pause button, speed selector (1×/2×/5×/20×), and timestamp display. The Output panel shows execution results from the selected point in time.*

Control	Description
▶ / ⏸ Play/Pause	Start or pause playback
Scrubber	Click or drag to jump to any point in the interview
Speed (1×/2×/5×/20×)	Control playback speed
Output panel	Shows the execution output that existed at the current replay timestamp
Timestamp	Shows the wall-clock time at the current replay position

---

5.3 Renaming a Session

By default, sessions are identified by their hex session ID. You can give a session a descriptive name:

1. Go to 📁 **Sessions**
2. Find the session
3. Click the ✏️ (pencil) button on the right side of the row
4. Type a name (e.g. "John Smith — Backend Round 2")
5. Press **Enter** or click **Save**

> ***Screenshot:*** *A session row with the rename input field open below the session details, showing a text input with a placeholder "Enter a name for this session..." and Save/Cancel buttons.*

- Names are saved instantly to localStorage
- To clear a name, open the rename input and save an empty value
- Named sessions show the name as a highlighted label in the row
- The search box searches session names too

5.4 Deleting a Session

1. Go to 📂 **Sessions**
2. Click the 🗑 (trash) button on the session row
3. A confirmation appears — click **Confirm delete** to permanently remove it

> ***Screenshot:*** *A session row in the delete-confirm state, showing "Confirm delete" (red button) and "Cancel" buttons replacing the normal action buttons.*

Deletion removes both the session metadata and all recorded snapshots. This action cannot be undone.

5.5 Searching Sessions

Use the search box at the top of the Sessions panel to filter sessions.

The search matches against:

- Session **name** (if renamed)
- Session **ID**
- **Language** (e.g. type "python" to show only Python sessions)
- **Code preview** (first 120 characters of the candidate's code at last update)

Click the ✕ button to clear the search.

---

6. Other Tools

6.1 Code Editor (Scratch Pad)

The `</>` **Code** tab gives you a full Monaco editor for your own use — write scratch code, test examples, or experiment with solutions yourself.

This editor is independent of any session. Running code here uses your own input, not the candidate's. Language and font size settings in the scratch pad sync with the candidate editor.

6.2 Whiteboard

The ✏️ **Whiteboard** tab opens a freehand drawing canvas.

> ***Screenshot:*** *The whiteboard with several hand-drawn boxes and arrows representing a system design diagram. The toolbar along the top shows color swatches, brush size, eraser, and clear button.*

Use it to:

- Sketch data structures or algorithms
- Draw system architecture diagrams during system design questions
- Work through a problem visually alongside the coding editor

The whiteboard is local — it is not shared with the candidate in real time.

6.3 Take-Home Mode

The 📋 **Take-Home** tab provides a standalone coding environment that does not require a live session.

Use it to:

- Send async coding problems to candidates who cannot do a synchronous interview
- Evaluate code submissions offline

- Test problems before sending them in a live interview

In take-home mode, candidates receive a link they can open at any time. There is no live monitoring — the interviewer reviews the submission after the candidate submits.

---

7. Supported Languages

Language	Runtime
JavaScript	Node.js
TypeScript	Transpiled via TypeScript compiler, then Node.js
Python	Python 3 (isolated mode, no user site-packages)
Java	OpenJDK (128 MB heap, 512 KB thread stack)
C++	g++ with `-std=c++17 -O2`
C	gcc with `-O2`
Go	`go run`
Ruby	ruby with `--disable-gems`
Rust	rustc
Bash	bash restricted mode
PHP	php with no php.ini extensions (`-n`)

Execution sandbox limits (all languages):

Limit	Value
Wall-clock timeout	10 seconds
Compile timeout (C/C++/Java/Rust)	30 seconds
Memory (virtual)	256 MB
CPU time	15 seconds
Max processes (fork-bomb protection)	64
Max open file descriptors	100
Max file write size	10 MB
stdout/stderr cap	256 KB

---

8. Troubleshooting

"Waiting for candidate to join..." — candidate has the link open

Cause: The candidate's browser hasn't successfully connected yet, or the polling hasn't detected their first keystroke.

Fix:

1. Ask the candidate to type a character in the editor and delete it — this forces a code update
2. Check that the candidate opened the **candidate link** (containing `role=candidate`), not the host link
3. If the issue persists after 30 seconds, ask the candidate to reload the page

---

Candidate's code stopped updating

Cause: Network interruption on either side, or the candidate's tab went to sleep (mobile/low-power mode).

Fix:

1. Check the **"last update"** timestamp in the status bar — if it's more than 30 seconds ago during active coding, there may be a connectivity issue
2. Ask the candidate to type something or reload their page
3. The session will resume automatically once connectivity is restored — no new session link is needed

---

"Run candidate's code" button is greyed out

Cause: Either no code has been received yet (`code` is empty) or a run is already in progress.

Fix: Wait for the candidate to write some code. The button enables automatically once code is present.

Code execution returns "Server busy"

Cause: The backend has hit its concurrent execution limit (20 simultaneous runs).

Fix: Wait 10–15 seconds and try again. This is a transient condition during peak load.

Code execution returns "Execution timed out"

Cause: The candidate's code ran for more than 10 seconds (infinite loop, slow algorithm, etc.).

Fix: This is expected behavior for code with bugs. The candidate should review their code for infinite loops or inefficient algorithms. The sandbox automatically kills the process.

Replay is missing or shows "No recording found"

Cause: One of the following:

- The interview was conducted in a **different browser** or on a **different computer**
- Browser storage was **cleared** (cookies/cache clear, private/incognito window, or browser data reset)
- The session is older than **180 days**

Fix:

- Recordings are stored in the interviewer's browser localStorage. Always conduct and review interviews in the same browser on the same machine.
- There is no server-side backup of recordings — if localStorage is cleared, recordings cannot be recovered.

> ***Best practice:*** *If you need a permanent record, use the* 📼 ***Replay*** *view and take a screen recording using your OS's built-in screen recorder (QuickTime on Mac, Xbox Game Bar on Windows).*

Candidate's window didn't close after "End Session"

Cause: Browsers block `window.close()` if the tab was not opened via `window.open()`. This is a browser security restriction.

Behavior: In this case, the tab navigates to a blank page instead after the 10-second countdown. The candidate's session is ended regardless — the blank page means they can no longer submit code.

Fix: No action required. The session is fully ended on the server side. The blank-page fallback is intentional.

"Session ID already taken" or session link not working

Cause: Session IDs are random 16-character hex strings generated in your browser. Conflicts are astronomically unlikely.

Fix: Click **Start Session** again to generate a new session ID.

The app is slow or unresponsive

Cause: The Railway backend (code execution server) may be cold-starting after a period of inactivity. Railway free-tier instances spin down when idle.

Fix: Click ▶ **Run** once (even with empty code) to wake the backend. The first request may take 5–15 seconds; subsequent requests are fast.

Questions panel is empty

Cause: The question bank is managed separately. If no questions appear, none have been added to the bank yet.

Fix: Contact your team's CoderPad administrator to add questions to the bank.

---

9. FAQ

Q: Does the candidate know they're being recorded? A: The candidate can infer the interview is being recorded (it is a monitored interview), but there is no explicit "recording" indicator in the candidate UI. Recording is a host-side feature — snapshots are saved in the interviewer's browser, not visible to the candidate.

Q: Can multiple interviewers watch the same candidate at once? A: Yes. Multiple people can open the same host link simultaneously. Each browser will independently poll for updates. However, only the browser that originally opened the host link will have session recordings saved locally.

Q: Can the candidate see the interviewer's cursor or edits? A: No. The session is one-directional for code: candidate → interviewer. The candidate only sees their own editor and any question description pushed by the interviewer.

Q: What happens if the candidate refreshes their page? A: Their code is not persisted server-side between page loads. If they refresh, they will need to re-type (or paste back) their code. Advise candidates

not to refresh during the interview.

Q: Can I use my own questions? A: Yes. The question bank can be extended by adding entries. Contact your administrator or the engineering team to add custom questions.

Q: How long are sessions stored? A: Session recordings are kept for **180 days** in your browser's localStorage, then automatically purged. The session ID link will stop working on Railway's server after the backend session expires (typically a few hours of inactivity), but the recording in your browser is independent of this.

Q: Is there a mobile/tablet version? A: The app works in mobile browsers but is optimized for desktop use. Coding interviews on a phone-sized screen are not recommended. Tablet-sized screens (iPad) work reasonably well.

Q: Can candidates install anything or is it all in-browser? A: 100% browser-based. No plugins, extensions, or downloads required. Any modern browser works.

Q: How secure is code execution? A: Each code execution runs in an isolated temporary directory with OS-level resource limits (memory, CPU, process count, file size). The environment has no network access and inherits no secrets. After execution, the temp directory is deleted. See the Security Summary in the README for full details.

Last updated: March 2026 · Platform: sigma-computing-interview-app.vercel.app